SaltBench v1: A Referee-Gated, Pre-Registered Protocol for Measuring Method Effects in Machine-Checked Software Work, with Frontier Baselines on Lean and Verus

Jason Hickey

September 2026
version 1, the protocol and the baselines; version 2 will carry the custom populations

Abstract

SaltBench is a benchmark protocol for one question: does a change to how a coding agent is instructed change what it can get past a machine referee? The protocol fixes four things before any model call. A referee decides every outcome (a proof kernel, a program verifier, or a hidden test suite), and the agent's work is re-checked outside its own elaboration. A fence denies the agent the network, the ground truth, and the harness state, and the fence is measured by probes before the run rather than assumed. Every run is authorized by a dated freeze commit, with predictions registered and scored, adverse outcomes named, and later changes appended as amendments that never edit the frozen text. A budget stop is a halt, never a failure. We report the protocol, two task populations (CLEVER Task 1 in Lean 4 and VeruSAGE-Bench proof targets in Verus), and frontier baselines. The plain arm saturates the hard band of the Verus population at Opus 5 (9 of 10, with the turn cap binding in the shoulder), and SWE-bench Verified at Sonnet 5 (13 of 15 on both control arms, the two failures at the call cap, with contamination present and unquantifiable). On the Lean population the pre-registered comparison of a plain arm, an equal-length placebo arm, and the treatment arm returned a null at Sonnet 5 (8 of 15 on every arm, and the treatment arm matched the control problem for problem), and a blind triage of the failures attributes most of them to the reference oracle rather than to the agent, including one problem whose reference specification is machine-checked unsatisfiable. We report the instrument findings as results in their own right: a cap measured from data the cap produced returns the cap; a gate can penalize the treatment for applying the treatment; a screen's false positives are invisible until someone goes back for the refused cell; a fence probed only in its sandbox's language cannot see the layer above it, and no scored episode of this campaign had the agent's own file tool fenced by path, though across 278 landed Lean episodes no agent ever directed a file tool at a fenced path. No effect of the treatment is claimed. The tests that could show one are stated and remain open.

1 Introduction

A benchmark for agent methods needs two properties that usually pull against each other. The outcome has to be decided by something the agent cannot argue with, and the comparison has to be planned before the agent runs, so that the result cannot be narrated afterwards. SaltBench is a protocol that insists on both. The referee is a proof kernel, a program verifier, or a hidden test suite, run outside the agent's own toolchain invocation. The comparison is pre-registered: the population is drawn by a committed seed, the arms are byte-pinned, the predictions are written

down, the adverse outcomes are named, and the freeze commit is the authorization. Everything that happens afterwards is a dated amendment appended to the record.

This paper is version 1. It claims the protocol, two populations with their provenance, frontier baselines on those populations, a pre-registered null on one of them with a triage that explains most of it, and a set of instrument findings that we think transfer to any benchmark of this kind. It does not claim an effect of the method under test. Section 6 states the tests that could produce one; they are the content of version 2.

The method under test is a set of working practices for machine-checked development that the author uses in a formal mathematics project. Its rendering as an arm is described in Section 2.6; only the part of it that a single sealed agent can carry is rendered, and that limit was registered before the first call.

2 The referee-gated protocol

2.1 The referee decides, outside the agent's elaboration

An episode ends when the agent stops or hits a cap. The file it leaves is then scored by a checker the agent never sees, on a machine the agent never reaches. What "solved" means is fixed per substrate.

Lean (CLEVER). The checker extracts only the agent-writable bodies by section markers, screens them for command-introducing and meta keywords with comments and string literals stripped, assembles a canonical file from the frozen statements and the bodies, and compiles it under an operating-system sandbox. It then replays every declaration of the compiled module through the Lean kernel (`Lean.Environment.replay`), so a declaration the elaborator admitted without the kernel is rejected. It compares, as kernel expressions, the type of every frozen declaration and the value of the human specification between the canonical file and a pristine file with `sorry` bodies, so a notation, macro, instance or `open` that changes what the frozen text means is caught by construction. It collects the axioms of the audited declarations and requires every set to be inside {`propext`, `Classical.choice`, `Quot.sound`}; `sorryAx`, user axioms, and `native_decide`'s `Lean.ofReduceBool` all fail here. The class names the first failing gate: `SCREEN`, `COMPILE`, `KERNEL_REJECTED`, `STATEMENT_ALTERED`, `AXIOMS_FAIL`, `PASS`.

Verus (VeruSAGE-Bench). Verus emits no proof object and there is no independent re-check, so the protocol says "the referee accepted" where the Lean protocol says "kernel". The integrity work that the kernel replay and axiom audit do in Lean is done here by three text and AST layers before the referee runs: a screen over the agent's regions; a count guard over `admit()`, `assume(`, `external_body` in both spellings, `assume_specification` and `verifier::admit`, canonical against original; and the benchmark's own `lynette additions` AST comparison. A screened or cheating body is never handed to the referee. `PASS` requires exit 0 and a results line that parses as `N verified, 0 errors` with $N \geq 1$ and no partial-verification suffix. Resource-limit exhaustion and compile errors are their own classes and are never charged to the arm.

SWE-bench Verified. The scorer is the unmodified `swebench` harness in the pinned evaluation images, with the fail-to-pass and pass-to-pass suites hidden from the agent. This substrate was abandoned after stage 0 (Section 4.1); its protocol is kept in the record.

2.2 The fence is measured, not assumed

The agent's subprocesses run under the operating system's sandbox (macOS Seatbelt through the agent harness's own sandbox settings): every host denied, writes only under the working copy and the session temp directory, reads denied on the harness state, the credential trees, and the run root. The checker runs under its own `sandbox-exec` profile. A command-text hook on the shell tool is kept as an attempt audit, not as the fence, and an audit layer records every tool read by path. Five smoke probes run before the driver starts: a canary, network escapes (including an in-language process spawn through the toolchain wrapper), reads of the arm files and the harness config, writes through the symlinked build directory, and the compile path. The driver refuses to start unless the last verdict of every probe is a pass carrying the freeze's episode-script hash.

Ground truth never reaches the host while an arm writes a specification blind. The stage-A views carry no human specification; the stage-B and stage-C views are shipped only after every stage-A episode has landed, and the ship is verified by a content set-hash rather than by the exit code of the tool that moved it.

Two findings in the record show why the fence is measured. The Verus referee's docstring claimed a sandbox, a process group and a belt; a grep of the file found only the comment, and the referee had been running unfenced. The exec set turned out to be four binaries, established by a ladder in which each arm's own error names the next binary, and a fifth exec point was revealed only by the red arm's panic text. A proof-only fixture had passed the four-binary fence, which would have been the receipt.

The third finding arrived after every read reported in this paper and applies to all of them. The sandbox's read denials bind sandboxed subprocesses: the shell tool and whatever it executes. The agent harness's own file-reading tool is executed by the harness process and never enters that sandbox. Neither substrate's settings carried a tool-permission rule, so no scored episode of this campaign had the agent's tools fenced by path. The smoke probe that had certified credential reads denied made its reads through the shell and through an in-language process spawn, both sandboxed; a probe written in the sandbox's language cannot see a hole in the layer above it, and it reads afterwards as though the agent itself were fenced. What was in force, each with its own evidence, was the subprocess sandbox, the empty network allowlist, the shell-tool hook, the ground-truth leak check, and the audit layer. That audit is what establishes the S2-Rust read: across its 15 Opus episodes no episode was voided and no unblocked read of a fenced path was recorded (one attempt through the shell was blocked). One episode of the lower-tier probe reported in Section 6 directed its file tool at a path under its own configuration directory, was not refused, and was voided as unscorable; read at the transcript the call returned that the file did not exist, so no bytes came back, and the audit field had recorded that the call was not blocked rather than that it was served. Those are different facts, and only one of them is a fence event. Over every landed S2-Lean episode in the campaign's four state roots, 278 with a transcript and 278 parsed, the count of file-tool calls at a fenced path is zero served and zero not served: no agent ever directed a file tool at a path under a deny root. The detector was driven on all three branches (a served read of a fenced path counted, a refused read classified and not counted, a read of an unfenced path ignored), because a quiet failure reads as good news. What this measures is what the agents did, not what they could have done: the canary shows the gap was open, and an absence of exploitation is not a presence of protection. The repaired fence derives a tool-permission deny list from the same path list as the sandbox denials, 132 rules beside 11 paths, and was driven with a canary in both arms: with the tool layer absent the canary reached the agent's final message, and with it present the agent reported the file unreadable and the canary appears in neither the result nor the transcript.

2.3 Pre-registration, and the amendment discipline

The dated freeze commit is the authorization. Before it: the population rule and seed, the arms as byte-pinned files, the checker, the constants, the reading rule, the predictions, the adverse outcomes, the stop rules with their enforcer. After it: nothing above the line is edited. Every later change, including a repair to an instrument, is a dated amendment appended before its own first model call, with its own predictions. Corrections to a registered document are appended as corrections, never edited away; the record carries several such corrections by its own authors.

The reading rule on the Lean and Verus populations is a count, not a p-value: for two arms on the same drawn problems, b is the number the first arm proves and the second does not, c the reverse, and $|b - c| < 5$ is read as INDISTINGUISHABLE. The threshold is registered before the first call and the outcome table is enumerated, with the adverse cell named. For the Verus wave the fixed separation is replaced by one derived from a measured null (a same-arm rerun), because the substrate's own paper reports that 11.8 % of failures flip on rerun.

Predictions are scored as they stand. Of the registered predictions in the record, several failed, and the failures are recorded as failures with their direction: the estimator over-priced five consecutive stages, then under-priced the sixth because one episode carried 64 % of a stage's spend.

2.4 A budget stop is a halt, never a failure

A per-episode token stop at four times the governing regime's p90 is registered as a halt: a halted episode is recorded as halted and is unresolved for the rate, because scoring an episode the budget stopped as a failure would let the budget instrument move the result. The multiple was chosen from the record: no passing episode among 177 had exceeded 2.40× its regime's p90, and the one runaway was at 7.79× and failed.

The rule had to be enforced where the verdict is written, not only where it is registered. The first halted Verus episode was scored `SCREEN` on a snapshot of an interrupted edit (two `assume` calls the agent had not yet discharged). The checker now takes the episode's termination and writes `class = HALT` for a token or wall stop, preserving the class it would otherwise have written as a diagnostic.

2.5 Ask of every gate which arm is more likely to trip it

The treatment arm encourages helper lemmas. Three gates in the Verus grader were found, in sequence, to refuse exactly that. The AST comparison refused a helper placed before the enclosing `impl`, classifying the episode as `STATEMENT_ALTERED`, a cheating class, for doing what the prompt invites; 47 of 207 views are impl-enclosed. The screen's rule set enforced 29 refusals of which the prompts named 12, and the unstated `use` rule bites in the helpers region; the repair makes the prompt a consumer of the rule set, with a gate that refuses a rule with no prompt entry. The helpers whitelist then split items by brace depth and refused a legitimate lemma whose `ensures` clause carried struct literals; it was the third episode ever run that found it, after 29 screen arms, 18 fence arms and an 11-arm fixture kit had passed. An instrument that penalizes the treatment for applying the treatment does not measure a small effect badly; it manufactures the opposite one. The discipline is now a standing question at every gate.

2.6 The arms

Every arm receives the same task file and the same stage prompt; only the agent's instruction file differs, and it is byte-pinned. `a0` is the plain arm: the base block alone. `a1` is the placebo: the

base block plus an equal-length process prompt with no method content (1,746 bytes against the treatment's 1,913; the base is 627). `a2` is the treatment: the base block plus the method's solo-renderable core. The rendering, article by article against its source, and the articles that are not rendered because they presuppose an organisation with a human in it, were registered before the first call; no result from `a2` may be read as a test of the method's multi-agent tier. The treatment text names `sorry`, which is the control's dominant failure; the objection that the arm coaches to the metric was registered in advance and is answered by the data in Section 4.2.

3 Populations and provenance

S2-Lean: CLEVER Task 1. `trishullab/clever` at commit `8348039`, MIT, Lean `v4.27.0`, mathlib `a3a10db0`; 161 problems. A raw problem file is re-cut by the harness into three views: stage A (write `generated_spec` from the docstring, ground truth absent), stage B (prove `spec_isomorphism` between the agent's frozen specification and the revealed human one), stage C (implement and prove `correctness` against the human specification, tests included). The draw sorts the real ids by a committed seed after removing the four structural exclusions of the benchmark's agentic-proving paper. Stage 0 ran $k = 30$. The registered comparison population is U15, the first fifteen unflagged ids of that order (the same paper flags 80 of 161 specifications), reached at depth 27. Stage C's population removes the two U15 ids whose stage-C view fails to elaborate before any agent text (problems 54 and 112) and one further id, problem 18, whose task is machine-checked unsatisfiable (Section 4.3), leaving 12.

S2-Rust: VeruSAGE-Bench proof targets. `microsoft/verus-proof-synthesis` at commit `cbf9c0c6`, MIT; the Anvil-Advanced and NRKernel projects, 267 records. The wave takes the unique `proof fn` target whose body is empty modulo whitespace: 207 records; the five refuse classes are counted and published with the draw. Stage 0 then ran every reference proof through the pinned referee inside the harness's own scaffold: 180 live, 27 with a task text that omits definitions its own ground truth supplies, zero dead tasks, zero pin defects, zero scaffold defects. The pin itself was measured: the current Verus release failed 5 of 15 reference proofs at the Rust front end, and the benchmark's own release passed 15 of 15; a front-end error on a reference file indicts the toolchain, never the task. The resource-limit curve is flat into the registered limit (179 of 180 at 10, 180 of 180 at 50 and at 250) and three repeats at the pinned seed produced zero flips.

S1: SWE-bench Verified. `princeton-nlp/SWE-bench_Verified` at revision `c104f840`, 500 rows, content-pinned by a hash over the canonicalised rows. The selection script is the normative form of the criteria; the draw is measured-50 and pilot-30 with a per-repository cap of 9, chosen from arithmetic over the eligible counts after the first draft's cap starved the draw in silence.

No task text from SWE-bench Verified is redistributed. Its dataset card carries no licence field, so rather than rely on a reading of what the issue text permits, the repository ships the 30 identifiers, the pinned revision, a hash over the 500 canonicalised rows and a hash over the 30-row projection the episodes read; `harness/fetch_problem_statements.py` rebuilds that projection from the pinned revision and verifies it against both. The rebuild was compared byte for byte against the file the episodes read before that file was removed from the tree.

Licences and attributions for every source, and the exact list of what this repository redistributes from each, are in `PROVENANCE.md`.

4 Results

4.1 SWE-bench Verified saturates at Sonnet 5 and is abandoned as a substrate

Stage 0 ran the two control arms on the first 15 tasks of the pilot draw at `claude-sonnet-5`, effort high, 40-call cap, in the pinned images. Pre-flight and gold controls passed 15 of 15 each; nothing was excluded.

arm	solved	metered p50	metered p90
`a0` plain	13/15	566,108	1,602,434
`a1` placebo	13/15	531,692	1,604,555

Both arms resolved the same 13 and failed the same 2; $b = c = 0$. The two failures are the two tasks on which both arms hit the 40-call cap (cap-bound episodes: plain 3, placebo 2), so 13 of 15 is a lower bound at this cap, as the Verus band's 9 of 10 is. Five of fifteen pairs shipped byte-identical patches across arms. The pre-registered contamination proxy (edit similarity of the submitted patch to the gold patch, cut at 0.80, stated before computing) read 6 of 13 resolved tasks as high-similarity, which the rule classifies as INDETERMINATE. An unregistered blind panel found, and the record verifies at the transcript, that on two unresolved tasks the agent wrote lines of the upstream fix as its own edit input before any read could have shown them; recall did not deliver either solve. The substrate is saturated at this tier for this scaffold, contamination is present and unquantifiable, and the treatment arms were never run on it. The datum stays in the record as the reason the campaign moved to referees that decide.

4.2 S2-Lean: a pre-registered null, a tier that moves both arms, and a placebo that moves with them

The control read. The plain arm at `claude-sonnet-5` on the first 15 of the draw at a 40-call cap proved the isomorphism on 2 of 15. Eleven of the thirteen failures compiled, replayed, kept their statements, and closed the proof with `sorry`. The benchmark's own reference checker, run over the same fifteen artifacts, agrees with ours on every problem (2 of 15). Raising the cap to 100 calls on the three round-capped episodes flipped two to proofs, one at 93 calls. On the registered U15 at a uniform 100-call cap the plain arm proved 8 of 15; the nine flagged problems in the first 15 of the draw were 0 of 9 at the control read's 40-call cap, and every one of them terminated voluntarily with turns in hand, so budget does not explain the zero. The flagged subset was not rerun at 100 calls, so the flagged against unflagged contrast is 0 of 9 at 40 against 8 of 15 at 100, with the better n on one side only.

The comparison. All three arms on U15 at `claude-sonnet-5`, 100 calls, stage A then stage B, arms alternating per problem:

contrast	b	c	reading
`a2` vs `a0`	0	0	INDISTINGUISHABLE; the treatment matched the control's set on all 15
`a1` vs `a0`	1	1	INDISTINGUISHABLE

Every arm proved 8 of 15. Seven of the treatment arm's fifteen episodes ended with `sorryAx` in `spec_isomorphism`, the identical failure on the identical set as the control; being told in plain

terms that a placeholder is not a proof changed neither which problems were solved nor how they failed. Totals were flat (50.7M, 51.7M, 51.4M metered tokens for `a0`, `a1`, `a2`). The paired split was not: on the eight problems both arms proved, `a2` used 15.79M tokens against `a0`'s 20.76M ($-24\,\%$); on the seven both failed, 35.62M against 29.92M ($+19\,\%$). This is $n = 8$ and $n = 7$ and is reported as a paired observation, not an effect.

The tier. The same U15, the same checker, `claude-opus-5`, both content arms: `a0` 10 of 15 and `a2` 10 of 15, `a0`-only $\{112\}$, `a2`-only $\{4\}$, $|b - c| = 0$. The whole run cost 26.1M tokens against the Sonnet run's 108.9M; the three 100-call cap-runners that failed at Sonnet were proved at Opus in a fraction of the tokens (problem 141: 11,757,659 tokens failing, 180,396 passing). The tier raise is not monotone per problem: `a0` lost problem 4. The cost split's sign did not reproduce at Opus. One treatment episode at Opus had been refused by the pragma screen for a `set_option` line; a registered differential re-check over the frozen artifact found a complete kernel-checked proof with and without the line, so `a2` reads 11 of 15 as a diagnostic and the contrast becomes $|b - c| = 1$, still INDISTINGUISHABLE.

The placebo at Opus proved 9 of 15, a strict subset of the plain arm's ten, missing only 112. Its registered band for arm-independence was $[9, 11]$; it landed on the boundary, by one problem's margin, and the reading is that the tier's gain is a property of the tier and not of prompt content.

Stage C at the ceiling. On the registered twelve stage-C problems at `claude-opus-5`, the plain arm certified implementation and correctness proof on 12 of 12, at a median of 284,716 tokens and 12 calls per episode, in 57 minutes. The registered gate reads a count of 8 to 12 as a ceiling hold; at 12 the reachability bound $c \leq n - P_0$ is $c \leq 0$, so there is no problem left for a treatment arm to win and the treatment did not run. A registered probe of the four problems whose reference oracle is provably permissive found that not one of the twelve passes exploited the vacuous region.

4.3 The triage: most of the null belongs to the oracle

A blind triage of every failed stage-B cell in the record (31 cells: 21 at Sonnet across three arms, 10 at Opus across two) labelled each from the two specification texts and the proof alone, with tier, arm, class and termination withheld. Thirty of the 31 cells are triageable: 17 are HUMAN-LOOSE (the agent's specification is strictly stronger than the human one, with a kernel-checked witness of non-isomorphism recorded where one could be written), 5 are AGENT-WRONG, 8 are PROOF-HARD. The thirty-first was a screen refusal that never failed to prove anything (the cell of amendment 9) and sits outside the taxonomy. The fourth registered label, HUMAN-BUGGY (the agent's specification contradicts a flagged clause), has count 0 and is unreachable on U15 by construction, because U15 excludes the flagged ids. The label is a property of the problem: eight failing problems, eight single labels, no exceptions, so the arms fail for the same reason on the same problems.

The pattern under four of the five false obligations is one defect: the human specification guards its conclusion with a well-formedness hypothesis and says nothing outside it, while the agent's specification, asked for as a `Prop` over the same signature, is total. A guarded specification and a total one are never isomorphic, and stage B asks for an isomorphism.

Problem 18 is worse than loose. In Lean `v4.27.0`, `String.drop` and `String.take` return a `String.Slice`, and slice equality is structural, so the reference specification's occurrence test is false at a genuine occurrence and the specification forces the answer 0 where the benchmark's own test demands 3. The statement that no implementation satisfies the specification and passes the test is machine-checked with axioms exactly the allowlist and no `sorry`. The elaboration check that guards the population had marked the problem fine, because it measures whether the task can

be stated, not whether it can be done. Within the registered population, problem 18 is the only carrier of this defect.

The audit of the benchmark's reference checker adds two findings. On non-adversarial artifacts it agrees with ours exactly. On adversarial ones it does not: its submission path rebuilds the problem and then compiles the solver's own view, so replacing the theorem to be proved with `True` certifies 13 of 15 with the solver's helper lemmas and 15 of 15 without them; and a helper lemma `axiom cheat (P : Prop) : P` discharging the real goal certifies 15 of 15, because an axiom is not a `sorry` and the checker greps for the `sorry` warning. The axiom audit and the statement comparison in our checker exist for exactly these two routes.

4.4 S2-Rust: the plain arm saturates the hard band at Opus 5

Pilot. Three tasks drawn from the 180 live at seed 20260902, plain arm, `claude-opus-5`: 3 of 3 passed, metered 76,003 / 466,390 / 612,390, wall 1.1 to 4.8 minutes. The third pass is a re-score: the grader's helper whitelist refused the episode's legitimate helper lemma (the third arm-correlated instance, Section 2.5), the landing file records that refusal, and the verdict was re-scored from the archived agent output. Three of four registered predictions failed, all in the direction of over-pricing; a large task file is not a large task.

The hard band. Because the pilot's three tasks straddled the middle tercile and all passed, the upper tercile of the 180 live tasks by reference-proof wall (rank ≥ 120; the cutpoints are ranks, because absolute walls re-time 2.2 to 2.8 times faster on an idle machine while the order is stable) was registered as the band, with its 13 drawn ids, before any of it was spent on. In this benchmark difficulty and size are nearly the same axis: Spearman $\rho = 0.892$ between reference-proof wall and task bytes over the 180 live, and 42 of the 60 upper-tercile tasks are Anvil-Advanced.

With a registered sequential early stop, 10 of the 13 were run: 10 scorable, zero void, and **9 of 10 passed** at a 40-call cap; the stop fired at 10 because the ceiling threshold of 9 was already reached, saving 10,459,778 tokens at no loss of information. Spend was 22,424,889 against a 32,884,667 quote. The registered prediction was 5 to 9 with a point estimate of 7; the band held at its top edge and the point estimate was low by 2, the fifth consecutive under-estimate of the plain arm.

The cap sits in the shoulder. Uncensored passing call counts were 23, 26, 27, 27, 32, 35, 35, 39 against a cap of 40; two of ten episodes were at the cap, one of them a pass at exactly 40, and the one failure died at 40 calls reading eight verified and one error. A p90 computed from data the cap produced returns the cap. The measured 9 is therefore a lower bound on the ceiling, and the room for a treatment arm is smaller than the count shows. That a cap of this size can cut an episode which would otherwise have passed is now evidenced directly rather than inferred from the shoulder: at the lower tier on this same band, an episode that reported no discharged obligations at the 40-call cap carried a complete verified proof at call 79 when the cap was raised (Section 6).

The median episode spends 81 % of its tokens before its first clean referee run (range 64 to 91). Cost is dominated by search, not verification; a cap cuts search, not polish.

5 The instrument findings, as results

Each of these was found by a gate, a probe, or a re-check in the record, and each changed the protocol.

1. **A censored cap returns the cap.** Section 4.4. Any budget derived from a capped distribution has to come from an uncensored read, or from a cap raised until the censoring fraction is near

zero. Recording the state each episode had reached when the cap cut it does not rescue the estimate on its own, because under a verifier that state can carry no information: an obligation is discharged or it is not, so an incomplete proof reports zero discharged until the moment it reports all of them. A partial count is a distance and a zero count is an absent measurement, and Section 6 reports an episode that read zero at the cap and held a complete verified proof thirty-nine calls later.

2. **A statistic is a cap's price only in the cap's unit.** The quota cost per token at the higher tier was roughly double, while the token count fell; a campaign that priced the tier on tokens alone would have called it free. The metered sum and the quota unit are different quantities and a stop rule has to say which it uses.
3. **A gate can penalize the treatment for applying the treatment.** Section 2.5, three instances in one wave, the third inside the layer added to prevent the first.
4. **A screen's false positives are invisible by construction.** The campaign ran 201 scored Lean episodes and examined exactly one refusal, because exactly one existed; it was a complete proof. A defence-in-depth layer needs its own false-positive reading routinely.
5. **A blind agent that passes is a measurement of a different task.** A fence derived from the run root denied the agent its own episode directory, so the referee wrapper was unreachable; the agent wrote proofs it could never check, and two of them passed the referee at 11 to 16 times the cost of a sighted episode. The episode driver now refuses when the fence and the workspace intersect.
6. **A deny list is only as broad as the layer that enforces it.** Section 2.2. The sandbox's path denials fenced the agent's subprocesses and not the agent's own file tool, and the probe that certified the fence made its reads through the sandbox, so it could not see the layer above. A fence has to be probed in the language of every tool it is meant to bind, and the probe has to be neutral: a probe that told the agent a tool was denied and asked it to route around was answered by the agent's judgement with no tool call made, and read as blocked while measuring nothing. The audit built to find such a hole in the record had the same shape of defect: its field recorded that a call was *not blocked*, which conflates a denial, an absent file and a served read, and it reported an episode as an escape whose read had returned that the file did not exist. Only a served read is a hole being used, and a field that cannot say which of the three it saw will report the wrong one.
7. **A gate that extracts by delimiter measures the delimiter.** A statement-immutability failure was reported as the agent altering the specification; the specification was byte-identical, and the mechanism was Lean naming a cached auxiliary `match` declaration after whichever declaration elaborated it first. The audit now compares matchers by content.
8. **A guard whose predicate cannot hold looks like a guard in every review.** The recall instrument's provenance guard compared a wrapper's hash to its wrappee's and had taken the fallback on 78 of 78 rows; the fallback read the same bytes, so no number moved. The token ceiling had been blind on every episode where it could have mattered, because the watchdog's call crashed on the first path-carrying tool call and the shell turned the crash into zero; repaired with a self-test that makes the caller's exact call.
9. **A counterfactual about someone else's instrument must be run against their code.** Section 4.3: the headline that the reference checker was sorry-inflated was refuted by its own kill-check, and what survived was a different hole.

6 The treatment question, open

Nothing in this record shows an effect of the treatment. On S2-Lean the treatment arm matched the control's set at Sonnet and was within one problem of it at Opus, with the placebo moving with the tier; the triage attributes most of the shared failures to the reference oracle. On S2-Rust and on stage C the plain arm is at or near the ceiling of every band the campaign can afford, so a comparison has no room. The question is open, and the next tests are:

1. **A lower tier on the hard band.** Registered as both arms at `claude-sonnet-5` on the same 13 Verus ids with the same early stop, quoted from a Sonnet probe on the band rather than from the Opus figures. The quote landed; the read was withdrawn; and the reason it was withdrawn is the result reported here. The probe paired three of the band's tasks across the two tiers, the same task and arm and instrument with only the model differing, at the same 40-call cap. Its aggregate paired token ratio was 1.55 (per-episode median 2.06), which prices the lower tier at 0.71 of the higher tier's per-episode quota and passes the affordability gate. But Sonnet passed 0 of the 3 where Opus passed 2, and all three Sonnet episodes ended at the cap, so a 13-id read at that cap would have been in part a measurement of the cap. One further episode was run to separate the two: the same task, the same tier and the same arm, at a cap of 120 calls instead of 40.

model	cap	calls	metered tokens	wall (s)	referee at the end
`claude-opus-5`	40	23	957,722	307	9 verified, 0 errors (pass)
`claude-sonnet-5`	40	40	3,051,144	768	0 verified, 1 errors (stopped at the cap)
`claude-sonnet-5`	120	79	8,375,623	1071	10 verified, 0 errors (pass)

   The 120-call episode passed cleanly: no screen violation, no helper-shape violation, the linter at return code zero, the count guard unchanged, the resource limit inside budget, the episode not void and no unblocked read of a fenced path. The cap was therefore binding at this tier on this task, and the identical Sonnet episode's report of no discharged obligations at call 40 was not a measure of how far it had to go. The planned 13-id read at 40 calls does not run, because it would return the cap. A read at 120 is a different and much larger commitment: one episode at 8.4 million tokens prices 26 episodes at roughly 218 million, and that figure is a lower bound, since the episode it is priced from passed at 79 of its 120 calls on the task the higher tier found easiest of the three. The two-tier comparison is therefore open and belongs to version 2. What the pair does establish is narrower, and it is one episode against one: on this task the lower tier reaches the same referee verdict as the higher one, and pays 8.7 times the tokens and 3.4 times the calls to get there.
2. **A CLEVER variant with the oracle repaired.** Replace the isomorphism obligation with an implication-with-witness (the agent's specification implies the human one, and the reference implementation satisfies the agent's), exclude problem 18, and rerun the seven problems both arms failed. This changes what the benchmark measures and is registered as a variant, not as a correction to the published Task 1 numbers.
3. **Populations with room.** Version 2 adds a custom population of classic systems components with textbook specifications, graded by size a priori and frozen (specification, tests, satisfiability witness, one specification perturbation for a maintenance arm, mutant set) before any model call, with every result reported. A scout for a second population over a contamination-free Rust software-engineering set returned no: of its 113 hand-authored tasks 5 are Rust, below the registered threshold of 8 before any screening, and none of the 5 survived the screen for a specification layer. The custom population ships in the Harbor task format, so that the referee, the fence and the task remain one object.

The failure surface on S2-Lean is also a finding about the benchmark: four of the twelve remaining stage-C oracles are permissive on regions their tests never visit, and the four-label triage taxonomy has no cell for a reference specification that contradicts its own tests. The reproduction scripts for both are in the repository, for the benchmark's authors; the disclosure itself is a separate act and is not claimed here.

7 Reproducibility

Every pin is a line in `harness/HASHES.txt`: the CLEVER commit, the Lean toolchain and mathlib revision, the project manifest and lakefile hashes, the Verus release and its rust channel, the z3, vstd and lynette hashes, the resource limit and seed, every prompt, arm file, checker and driver, and the set-hash of the 207 rebuilt Verus views. The evaluation images for the SWE-bench substrate are pinned by digest in `IMAGE-DIGESTS.json`. The episode script refuses to run a stage whose view or checker hash is not the pinned one, and each manifest records the freeze commit, the hashes it asserted, the model id read from every response, and the transcript's hash. The morning-line instruments that produced every rate in this paper are in the repository with their self-tests, and the evidence directories carry their outputs byte for byte.

The task populations are re-derived from the pinned sources by the harness's view builders; the repository redistributes only what `PROVENANCE.md` lists. The run records and the S2 episode archives are a separate data asset; its DOI is assigned at release (Zenodo) and recorded in the repository on the flip day. Version 2's populations ship as Harbor tasks so that the referee, the fence and the task are one object.

The code is released under Apache-2.0 and the data and documents under CC BY 4.0. The agent was Claude Code on a consumer subscription, and the transcripts are published as this campaign's own measurements: the provider's Consumer Terms effective 2025-10-08 assign outputs to the user and do not restrict their publication, and the training restriction in the Usage Policy dated 2025-09-15 binds the subscriber rather than a recipient of these files. Both documents are cited at the version read, because both are revised in place.

Acknowledgements

CLEVER is by the Trishul group at UT Austin (MIT licence); VeruSAGE-Bench and lynette are by Microsoft (MIT licence); SWE-bench Verified is by the SWE-bench authors with OpenAI's verification pass; the source repositories of the drawn issues carry their own licences, listed in `PROVENANCE.md`. The runs were executed by Claude Code with Anthropic's Sonnet 5 and Opus 5 models. The protocol documents, amendments and result files in the repository were written by the author's assistant heads under the author's direction and carry the author's responsibility.

A S2-Lean U15, per problem

problem	a0 S	a1 S	a2 S	a0 O	a2 O	a1 O
73	F	P	F	P	P	P
0	F	F	F	F	F	F
146	P	P	P	P	P	P
16	P	P	P	P	P	P
4	P	F	P	F	P	F
38	P	P	P	P	P	P
142	P	P	P	P	P	P
96	F	F	F	F	F	F
112	F	F	F	P	F†	F
141	F	F	F	P	P	P
31	P	P	P	P	P	P
54	P	P	P	P	P	P
127	F	F	F	F	F	F
18	F	F	F	F	F	F
74	P	P	P	P	P	P
proven	8	8	8	10	10	9

S = `claude-sonnet-5`, O = `claude-opus-5`, stage B, 100-call cap. † refused by the pragma screen; a registered re-check over the frozen artifact found a complete proof (diagnostic 11 of 15).

B The stage-B failure triage

problem	label	ground, in one line
0	HUMAN-LOOSE	conclusion guarded by `numbers.length > 1`; vacuous on short lists; witness recorded
4	HUMAN-LOOSE	guarded by `0 < numbers.length`; vacuous on `[]`
18	AGENT-WRONG	agents scan `List.range (|s| - |t| + 1)`, contradicting a `#test`; the human spec is also unsatisfiable
73	PROOF-HARD	equivalent specs; the equivalence needs an optimality argument
96	HUMAN-LOOSE	spec fixes membership only, never order or multiplicity; witness `primes ++ primes`
112	PROOF-HARD	true isomorphism; one sibling cell proved it completely
127	HUMAN-LOOSE	guarded by interval well-formedness; vacuous on ill-formed intervals
141	PROOF-HARD	reduces to a `String.splitOn` equivalence; 7 to 8 kB of lemmas written and still out of rounds

Totals over 30 triageable cells: HUMAN-LOOSE 17, AGENT-WRONG 5, PROOF-HARD 8; one screen-refused cell excluded.

C The S2-Rust hard band

Upper tercile of the 180 live tasks by reference-proof wall, rank ≥ 120; band $n = 60$, Anvil-Advanced 42 and NRKernel 18; band wall median 4.55 s, max 358.60 s; band bytes median 165,568. Draw of 13 at seed 20260902: 11 Anvil-Advanced, 2 NRKernel. Ten run under the sequential stop: 9 `PASS`, 1 `VERIFY_FAIL` at the cap; sighted episodes reach the first clean referee run at a median of 2 referee calls (p90 9). The 9 is a lower bound on the band's pass count at this cap: the failed episode ended

at 40 of 40 calls one obligation short (8 verified, 1 error), the uncensored passing call counts run 23 to 39 against the cap of 40 with p90 = 39, and one pass landed at exactly 40 of 40.